UNIVERSIDAD NACIONAL DE LOJA

FACULTAD AGROPECUARIA Y DE RECURSOS NATURALES RENOVABLES

CARRERA DE INGENIERÍA AMBIENTAL

Período Académico: Septiembre 2026 – Febrero 2027 | Modalidad: Presencial

GUÍA PASO A PASO: REPOSITORIO GITHUB, VISUAL STUDIO CODE, ENTORNO VIRTUAL PYTHON (venv) Y BASES DEL PROYECTO INTEGRADOR

MATEMÁTICA II: CÁLCULO INTEGRAL APLICADO A LA INGENIERÍA AMBIENTAL

Docente: Dr. Guillermo Chuncho | *Repositorio: https://github.com/cguillermo79/Informacion_matematica*

1. Propósito y Objetivos de la Guía

Esta guía técnica tiene como finalidad orientar de manera detallada, secuencial y reproducible a los estudiantes de la Carrera de Ingeniería Ambiental en la configuración integral de su espacio de trabajo en sus computadoras personales.

Objetivos de Aprendizaje Operativo

Al finalizar esta guía, cada estudiante y equipo de trabajo estará en capacidad de:

1. Instalar y verificar las herramientas base de desarrollo científico: Git, Python 3 y Visual Studio Code.
2. Clonar el repositorio oficial de GitHub del curso y mantenerlo actualizado localmente.
3. Crear, configurar y activar un **entorno virtual aislado de Python (`venv`)** para garantizar que los scripts y librerías funcionen sin colisiones de versiones.
4. Vincular correctamente el intérprete virtual en Visual Studio Code.
5. Acceder a las bases de datos geoespaciales y climáticas asignadas a su grupo (ventanas de estudio de 400 o 100 celdas en las zonas Norte/Sur y la serie cantonal de incendios del Cantón Loja).
6. Conocer la estructura de carpetas donde desarrollarán y depositarán los entregables de la Unidad 1, Unidad 2 y Unidad 3, y el flujo de comandos en Git para sincronizar su trabajo.

2. Requisitos Previos: Instalación del Software Base

Antes de ejecutar los comandos en la terminal, asegúrese de tener instalado en su computadora el siguiente software gratuito y de código abierto:

1. **Git (Sistema de Control de Versiones):**
   - Descargar desde el sitio oficial: `https://git-scm.com/downloads`.
   - Durante la instalación en Windows, deje las opciones por defecto recomendadas (asegúrese de permitir Git desde la línea de comandos de Windows).
2. **Python 3 (Versión 3.10, 3.11 o 3.12 recomendada):**
   - Descargar desde: `https://www.python.org/downloads/`.
   - **¡PASO CRÍTICO EN WINDOWS!:** Al iniciar el instalador, marque obligatoriamente la casilla inferior que dice: ***"Add python.exe to PATH"*** antes de hacer clic en *Install Now*. Si omite este paso, el comando `python` no será reconocido en la terminal.

3. **Visual Studio Code (VS Code):**
   - Descargar desde: https://code.visualstudio.com/Download.
   - Extensiones recomendadas dentro de VS Code (instalar desde la pestaña de Extensiones `Ctrl+Shift+X`):
     - **Python** (desarrollada por Microsoft).
     - **Jupyter** (para cuadernos interactivos `.ipynb`).
     - **GitLens** (para trazabilidad y control de ramas en Git).

3. Paso a Paso en la Terminal: Clonación y Apertura en VS Code

Abra la terminal de su sistema operativo: en Windows puede usar **PowerShell** o la **Terminal de Windows**; en macOS o Linux utilice la aplicación **Terminal**.

Paso 1: Configurar su identidad en Git (Solo si es la primera vez)

Abra su terminal y configure su nombre y su correo electrónico personal (el mismo asociado a su cuenta de GitHub) para que sus contribuciones queden registradas adecuadamente:

```
git config --global user.name "Nombres y Apellidos del Estudiante"
git config --global user.email "correo.personal@gmail.com"
```

Para verificar que quedó bien configurado, ejecute:

```
git config --list
```

Paso 2: Crear una carpeta de trabajo y ubicarse en ella

Se recomienda tener una carpeta ordenada en su disco local (por ejemplo en `Documentos`):

```
# En Windows PowerShell o CMD:
cd $HOME\Documents
mkdir Matematica_UNL
cd Matematica_UNL

# En Linux o macOS:
cd ~/Documents
mkdir Matematica_UNL
cd Matematica_UNL
```

Paso 3: Clonar el repositorio oficial del curso desde GitHub

Ejecute el comando `git clone` utilizando la dirección URL pública del repositorio del curso:

```
git clone https://github.com/cguillermo79/Informacion_matematica.git
```

Una vez que finalice la descarga de todos los archivos y bases de datos, ingrese a la carpeta del proyecto:

```
cd Informacion_matematica
```

Paso 4: Abrir el proyecto en Visual Studio Code

Desde la misma terminal, dentro de la carpeta `Informacion_matematica`, ejecute:

```
code .
```

Nota: El punto (`.`) indica que se debe abrir la carpeta actual. Si el comando `code` no abre VS Code, abra manualmente Visual Studio Code, vaya al menú superior: **File** → **Open Folder...** y seleccione la carpeta `Informacion_matematica`.

4. Creación y Activación del Entorno Virtual de Python (venv)

¿Por qué es indispensable crear un entorno virtual?

Un entorno virtual (`venv`) es una instalación aislada y autónoma de Python dentro del proyecto. Evita conflictos de paquetes entre diferentes asignaturas, garantiza que todas las librerías necesarias (`pandas`, `numpy`, `scipy`, `matplotlib`) tengan las versiones exactas y no modifica la configuración global de su sistema operativo.

Realice todos los siguientes pasos desde la **Terminal Integrada de Visual Studio Code** (puede abrirla pulsando las teclas `Ctrl + ~` o `Ctrl + Shift + ~`, o en el menú superior: **Terminal** → **New Terminal**):

Paso 5: Crear el entorno virtual en la raíz del proyecto

Estando en la terminal dentro de `Informacion_matematica`, ejecute:

```
# En Windows (PowerShell o CMD):
python -m venv .venv

# En macOS o Linux (o si tiene multiples versiones instaladas):
python3 -m venv .venv
```

Esto creará una subcarpeta oculta llamada `.venv` en su proyecto con el intérprete de Python y su propio gestor de paquetes `pip`.

Atención en Windows PowerShell: Error de Políticas de Ejecución

Si al intentar activar el entorno virtual en Windows PowerShell le aparece el error: *"File ... Activate.ps1 cannot be loaded because running scripts is disabled on this system"*, debe habilitar la ejecución de scripts para su usuario ejecutando **una sola vez** este comando en la terminal:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Presione **Y** (Sí) cuando el sistema le pida confirmación.

Paso 6: Activar el entorno virtual

Ejecute el comando correspondiente a su terminal y sistema operativo:

```
# En Windows (PowerShell):
.\.venv\Scripts\Activate.ps1

# En Windows (CMD - Simbolo del sistema):
.\.venv\Scripts\activate.bat

# En macOS y Linux (Bash o Zsh):
source .venv/bin/activate
```

¿Cómo saber si el entorno virtual está activo?
En la terminal observará que al inicio de la línea de comandos aparece el prefijo `(.venv)` entre paréntesis, por ejemplo:

```
(.venv) PS C:\Users\Estudiante\Documents\Matematica_UNL\Informacion_matematica>
```

Paso 7: Seleccionar el Intérprete de Python dentro de Visual Studio Code

Para que VS Code y los cuadernos de Jupyter reconozcan el entorno virtual:

1. Presione en su teclado la combinación `Ctrl + Shift + P` (o `Cmd + Shift + P` en Mac).
2. Escriba: `Python: Select Interpreter` y presione Enter.

3. En la lista desplegable, elija la opción recomendada que contiene `(.venv)`: `.venv\Scripts\python.exe` (o `.venv/bin/python`).

En la esquina inferior derecha de la ventana de VS Code verá reflejado `3.1x.x ('.venv': venv)`.

Paso 8: Actualizar pip e instalar las librerías científicas requeridas

Con el entorno virtual activo `(.venv)`, actualice el gestor `pip` e instale todas las librerías científicas del curso ejecutando:

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Si no tuviera el archivo `requirements.txt`*, puede instalar las librerías directamente con:*

```
pip install numpy scipy pandas matplotlib seaborn jupyter pyarrow fastparquet
    openpyxl
```

5. Acceso y Verificación de las Bases de Datos por Equipo

Las bases de datos oficiales ya se encuentran descargadas en la carpeta `Unidad_1/bases_datos_proyecto_integrador/` (archivos CSV pequeños, de 5 KB a 1 MB). El significado de cada columna está en `DICCIONARIO_VARIABLES.csv`. Cada equipo debe identificar la subcarpeta asignada a su temática y zona de estudio:

Equipo	Bloque Temático Asignado	Zona	Carpeta Asignada	Dimensión de la Base
Equipo 1	Relieve y Accesibilidad	Norte	`equipo_01_relieve_norte/`	400 celdas (20×20), 400 reg, 17 col
Equipo 2	Relieve y Accesibilidad	Sur	`equipo_02_relieve_sur/`	400 celdas (20×20), 400 reg, 17 col
Equipo 3	Cobertura Vegetal y Suelo	Norte	`equipo_03_cobertura_norte/`	100 celdas (10×10) × 72 meses, 7.200 reg, 28 col
Equipo 4	Cobertura Vegetal y Suelo	Sur	`equipo_04_cobertura_sur/`	100 celdas (10×10) × 72 meses, 7.200 reg, 28 col
Equipo 5	Clima y Atmósfera	Norte	`equipo_05_clima_norte/`	100 celdas (10×10) × 72 meses, 7.200 reg, 22 col
Equipo 6	Clima y Atmósfera	Sur	`equipo_06_clima_sur/`	100 celdas (10×10) × 72 meses, 7.200 reg, 22 col
Equipo 7	Incendios y Severidad	Cantonal	`equipo_07_incendios_loja/`	Serie mensual (72 reg) y resumen por celda (7.997 reg)

Cuadro 1: Subcarpetas autorizadas para cada equipo de trabajo en el proyecto integrador.

Paso 9: Script de verificación rápida en Python

Para verificar que el entorno virtual lee correctamente la base de datos de su equipo, cree un archivo de prueba `test_datos.py` o abra un cuaderno de Jupyter y ejecute el siguiente código (modifique la ruta según su equipo):

```
import pandas as pd
from pathlib import Path

# Ejemplo para el Equipo 1 (Relieve Zona Norte):
carpeta =
    Path("Unidad_1/bases_datos_proyecto_integrador/equipo_01_relieve_norte")
ruta_base = carpeta / "equipo_01_relieve_norte.csv"

if ruta_base.exists():
    df = pd.read_csv(ruta_base)
    print("Lectura exitosa de la base de datos!")
    print(f"Filas totales: {df.shape[0]}, Columnas: {df.shape[1]}")
    print(f"Celdas espaciales unicas: {df['cell_id'].nunique()}")
```

```
    print(df.head(3))
else:
    print(f"Error: No se encontro el archivo en {ruta_base}")
```

6. Estructura de Carpetas de Entregables por Unidad

El repositorio cuenta con una estructura jerárquica estandarizada para las tres unidades del ciclo académico. Cada equipo debe colocar sus trabajos exclusivamente en las carpetas correspondientes:

```
Informacion_matematica/
|-- Unidad_1/
|   |-- avance_proyecto_integrador/
|   |   |-- avance_1/
|   |   |-- avance_2/
|   |   `-- avance_3/
|   |-- guias_de_estudio/
|   |   |-- guia_1/                    <- Guia general del proyecto (PDF y LaTeX)
|   |   |-- guia_2/                    <- Guia de entorno y GitHub (PDF y LaTeX)
|   |   `-- guia_3/                    <- Guia de integrales (PDF y LaTeX)
|   |-- lecciones/
|   |-- base_de_datos/                 <- Carpeta unica de bases oficiales
|   |   |-- Grupo_01_Relieve_Norte/
|   |   |-- Grupo_02_Relieve_Sur/
|   |   |-- Grupo_03_Cobertura_Norte/
|   |   |-- Grupo_04_Cobertura_Sur/
|   |   |-- Grupo_05_Clima_Norte/
|   |   |-- Grupo_06_Clima_Sur/
|   |   `-- Grupo_07_Incendios_Loja/
|   |-- defensa_individual/
|   |-- formulacion/
|   |-- guia/
|   `-- producto_tecnico/
|-- Unidad_2/
|   |-- avance_proyecto_integrador/    <- Contiene avance_1, avance_2 y avance_3
|   |-- guias_de_estudio/
|   |   |-- guia_1/                    <- Guia general del proyecto (PDF y LaTeX)
|   |   `-- guia_2/                    <- Guia de entorno y GitHub (PDF y LaTeX)
|   |-- lecciones/
|   |-- defensa_individual/
|   |-- formulacion/
|   |-- guia/
|   `-- producto_tecnico/
`-- Unidad_3/
  |-- avance_proyecto_integrador/    <- Contiene avance_1, avance_2 y avance_3
  |-- guias_de_estudio/
  |   |-- guia_1/                      <- Guia general del proyecto (PDF y LaTeX)
  |   `-- guia_2/                      <- Guia de entorno y GitHub (PDF y LaTeX)
  |-- lecciones/
  |-- defensa_individual/
  |-- formulacion/
  |-- guia/
  `-- producto_tecnico/
```

7. Flujo de Trabajo con Git: Actualizar y Subir Entregables

Para sincronizar el trabajo de su equipo y mantener al día las guías y materiales compartidos por el docente:

1. Descargar actualizaciones del docente (Antes de empezar a trabajar)

Cada vez que el docente publique nuevas guías o avisos en el repositorio oficial, ejecute en su terminal:

```
git pull origin main
```

Esto incorporará los nuevos archivos a su computador sin alterar su código local.

2. Registrar y guardar los avances de su equipo

Una vez que hayan elaborado su documento de formulación, script o informe en la carpeta correspondiente de su unidad:

```
# Verifique que archivos modifico o creo:
git status

# Agregue los archivos de su entrega (ejemplo para Unidad 1):
git add Unidad_1/01_formulacion_3pct/

# Guarde el commit con un mensaje claro y formal:
git commit -m "Equipo 01: Entrega Fase 1 Formulacion del Proyecto Unidad 1"
```

3. Subir sus cambios al repositorio remoto

Para enviar sus cambios a GitHub (si trabaja en una rama de equipo o en su fork):

```
git push origin main
```

Buenas Prácticas Obligatorias: Archivo .gitignore

Nunca suba a Git archivos pesados autogenerados ni entornos virtuales. El repositorio cuenta con un archivo `.gitignore` configurado para omitir automáticamente:

- La carpeta del entorno virtual: `.venv/`
- Cachés de Python: `__pycache__/`, `*.pyc`
- Puntos de control de Jupyter: `.ipynb_checkpoints/`
- Archivos temporales de compilación de LaTeX: `*.aux`, `*.log`, `*.out`, `*.toc`, `*.synctex.gz` (solo se suben el `.tex` y el `.pdf`).

8. Resumen Rápido de Comandos en Terminal (Cheatsheet)

Acción Requerida	**Comando Windows (PowerShell)**	**Comando Linux / macOS**
Clonar repositorio	`git clone <URL>`	`git clone <URL>`
Abrir carpeta en VS Code	`code .`	`code .`
Crear entorno virtual	`python -m venv .venv`	`python3 -m venv .venv`
Habilitar scripts (solo 1 vez)	`Set-ExecutionPolicy RemoteSigned -Scope CurrentUser`	No aplica
Activar entorno virtual	`.\.venv\Scripts\Activate.ps1`	`source .venv/bin/activate`
Desactivar entorno virtual	`deactivate`	`deactivate`
Instalar librerías del curso	`pip install -r requirements.txt`	`pip install -r requirements.txt`
Actualizar cambios remotos	`git pull origin main`	`git pull origin main`
Ver estado de cambios	`git status`	`git status`
Preparar archivos	`git add <carpeta o archivo>`	`git add <carpeta o archivo>`
Confirmar entrega local	`git commit -m "Mensaje"`	`git commit -m "Mensaje"`
Subir cambios al remoto	`git push origin main`	`git push origin main`

Cuadro 2: Referencia rápida de comandos de terminal para el proyecto integrador.